11강. 웹 서버 만들기

React

Express 프레임워크

- **Express 프레임워크란?**

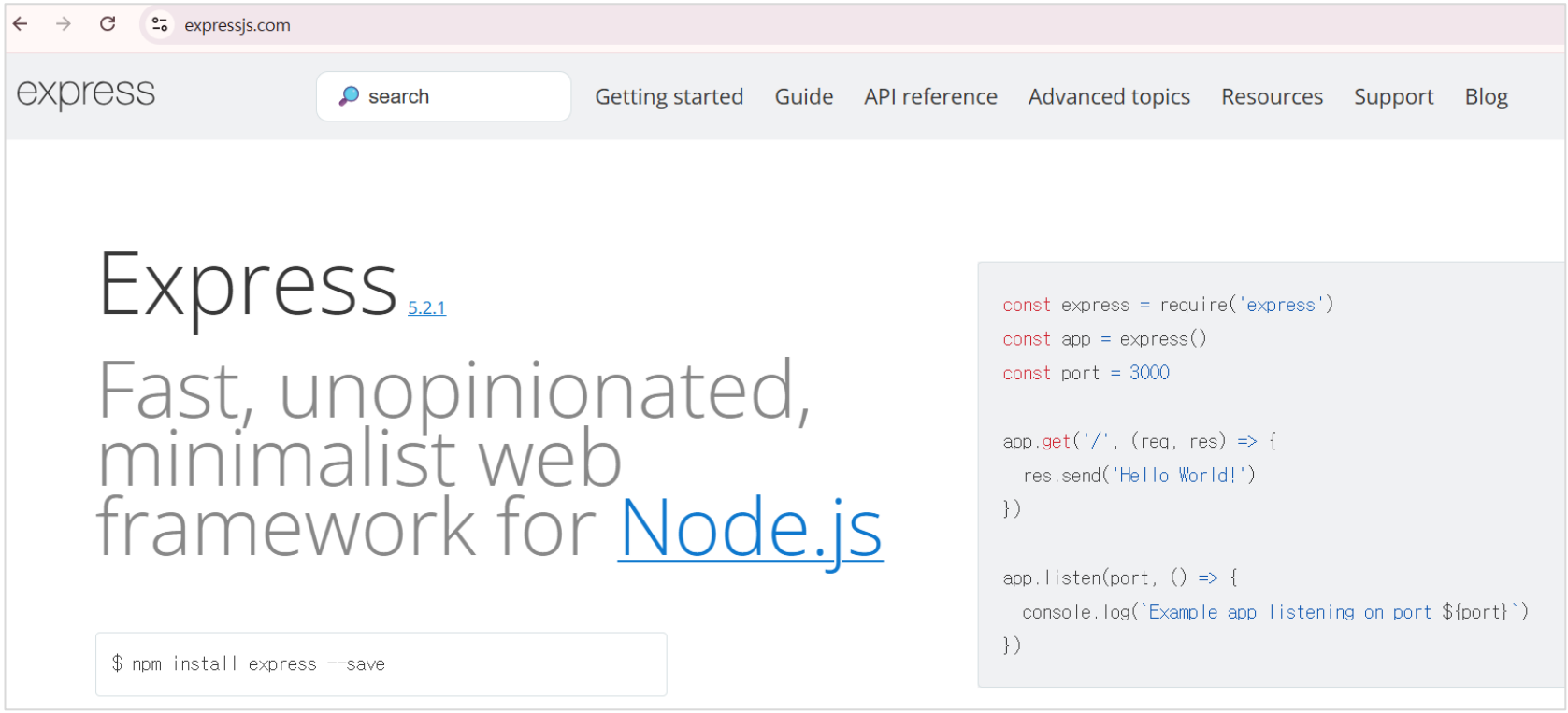
  Express는 **Node.js 기반에서 동작하는 웹 서버 프레임워크**로,
  서버 개발을 쉽고 빠르게 할 수 있도록 도와주는 **가볍고 유연한 백엔드 도구**입니다.

  - 사용하는 곳

  ✓ REST API 서버
  ✓ 프론트엔드(React 등) 백엔드
  ✓ 간단한 웹 서버
  ✓ 풀스택 프로젝트

Express 프레임워크

- **Express 프레임워크란?**



api 서버

- **프로젝트 개요**

프로젝트 이름 : api-server

깃허브 리포지터리: api-server

관련 주요 내용: README.md

개발 환경: node.js + express framework

api 서버

- **Express 프레임워크**

```
## 웹 서버 만들기
- Language: Javascript
- Node.js 설치
- package.json 생성: npm init -y

### Express 설치
- Express: Node.js 기반에서 동작하는 웹 서버 프레임워크
- 설치 : npm install express

### index.js 생성
- import express from 'express'
- 서버 코드 작성(PORT: 8000)
- 서버 실행 : node index.js
- 오류: ES6 문법 사용(import)
  package.json에 "type": "module"로 변경
```

README.md

api 서버

- 깃허브 리포지터리 연동 : api-server

kiyongee2 server run

.gitignore	server create
README.md	server run
index.js	server run
package-lock.json	server create
package.json	server run

README

웹 서버 만들기

- Language: Javascript
- Node.js 설치
- package.json 생성: npm init -y

api 서버

- **package.json**

```
{
  "name": "api-server",
  "version": "1.0.0",
  "description": "- Language: Javascript\r - Express: Node.js
  "main": "index.js",
  ▷디버그
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module",
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

import 구문 사용 가능

api 서버

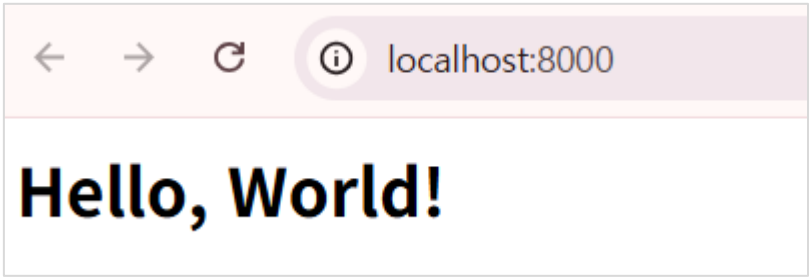
- **index.js**

```
import express from 'express';

// Express 객체 생성
const app = express();

// 라우트 설정
app.get('/', (req, res) => {
  // 클라이언트에게 응답 보내기
  res.send('<h2>Hello, World!</h2>');
});

// 서버 실행
const PORT = 8000;
app.listen(PORT, () => {
  console.log(`서버 실행중... http://localhost:${PORT}`);
});
```

api 서버

- **서버 실행**

```
PS D:\웹(js기반)\api-server\api-express> node index.js
서버 실행중... http://localhost:8000
```



api 서버

- **서버 자동 실행 – nodemon 설치**

```
npm i nodemon
```

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "start": "nodemon index.js"
},
"keywords": [],
"author": "",
"license": "ISC",
"type": "module",
"dependencies": {
  "express": "^5.2.1",
  "nodemon": "^3.1.14"
}
```

nodemon index로 실행

api 서버

- **상품 목록 API – index.js**

```
// 상품 목록 API
app.get('/products', (req, res) => {
  const products = [
    { id: 1, name: '무선마우스', price: 30000 },
    { id: 2, name: '유선키보드', price: 20000 },
    { id: 3, name: '웹캠', price: 40000 },
  ];
  res.json(products); // JSON 형태로 응답 보내기
});
```

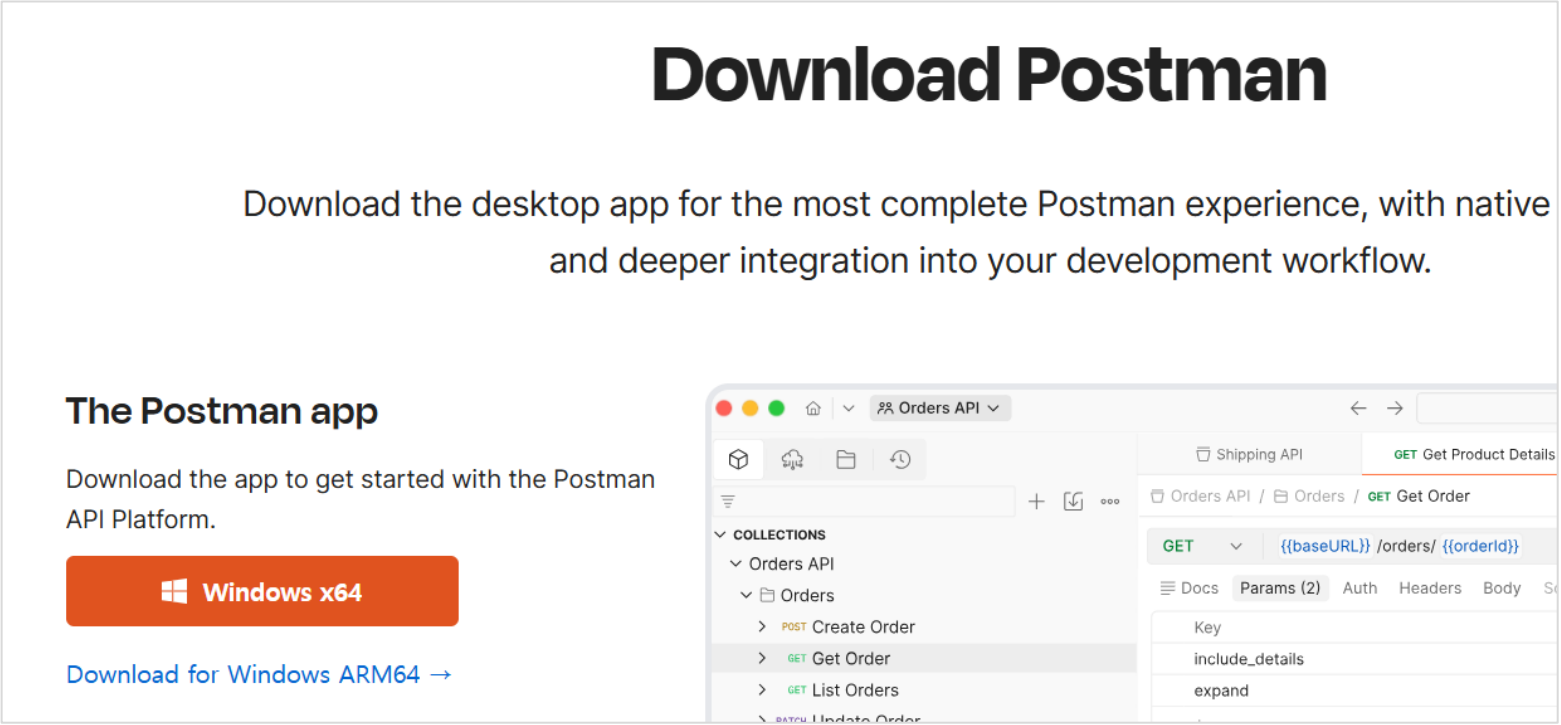
localhost:8000/products

pretty print 적용


```
[
  {
    "id": 1,
    "name": "무선마우스",
    "price": 30000
  },
  {
    "id": 2,
    "name": "유선키보드",
    "price": 20000
  },
  {
    "id": 3,
    "name": "웹캠",
    "price": 40000
  }
]
```

Postman – 테스트 도구

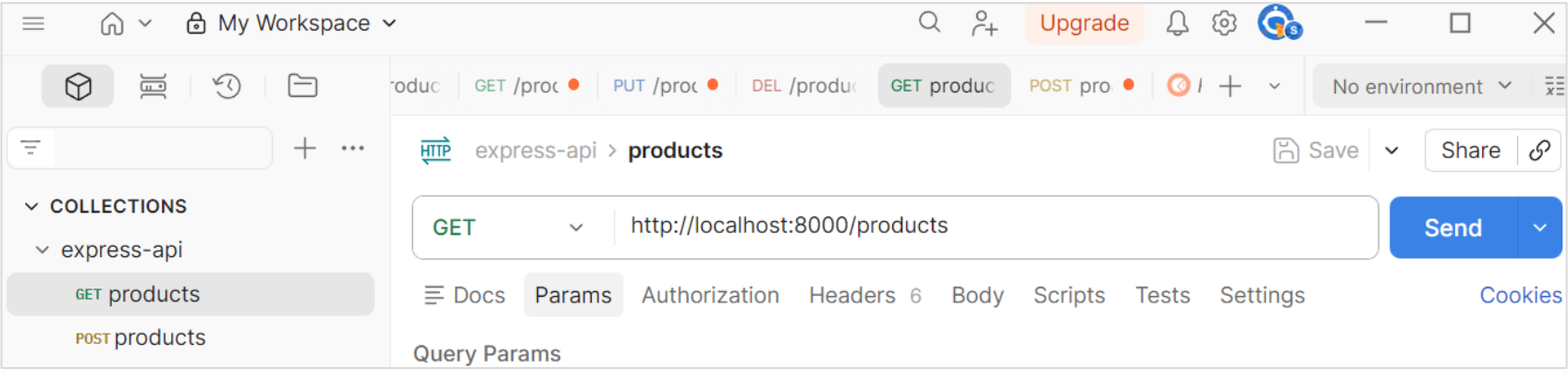
- **Rest API – 테스트 도구: postman 설치**



Postman – 테스트 도구

- **Rest API – 테스트 도구: postman 설치**

  My Workspace > COLLECTION > 추가



Postman – 테스트 도구

- **Rest API – 테스트 도구: postman 설치**

Body 200 OK

{} JSON Preview Visualize


```
[
    {
        "id": 1,
        "name": "무선마우스",
        "price": 30000
    },
    {
        "id": 2,
        "name": "유선키보드",
        "price": 20000
    },
    {
        "id": 3,
        "name": "웹캠",
        "price": 40000
    }
]
```

Postman – 테스트 도구

- **상품 등록 API – index.js**

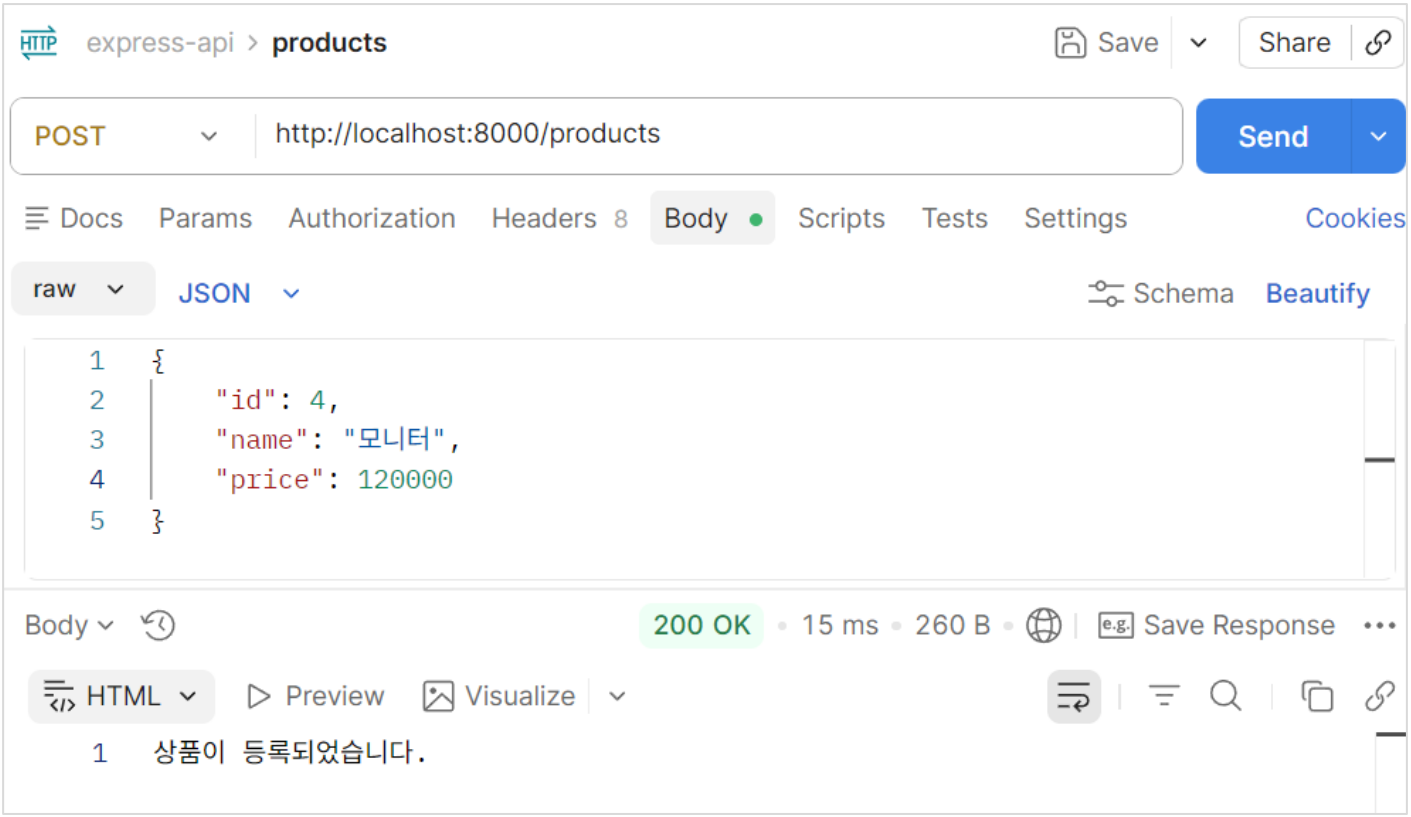
  express.json() 미들웨어를 등록해야 req.body에서 JSON 데이터를 읽을 수 있습니다.

```
// JSON 요청 본문 파싱 미들웨어
app.use(express.json());

// 상품 등록 API
app.post('/products', (req, res) => {
  // postmal 테스트 도구 사용해서 body에 json 형태로 데이터 전달
  console.log(req.body); //{ id: 4, name: '모니터', price: 120000 }
  res.send('상품이 등록되었습니다.');
});
```

Postman – 테스트 도구

- **Rest API – post 요청 보내기**

  Body > JSON



SQL이란?

SQL(Structured Query Language)

- '에스큐엘', 또는 '시퀄'이라 부른다.
- 사용자와 데이터베이스 시스템 간에 의사 소통을 하기 위한 언어이다.
- 사용자가 SQL을 이용하여 DB 시스템에 데이터의 검색, 조작, 정의 등을 요구하면 DB 시스템이 필요한 데이터를 가져와서 결과를 알려준다.

구분	개념
DDL(Data Defiintion Language) - 데이터 정의어	테이블을 포함한 여러 객체를 생성, 수정, 삭제하는 명령어
DML(Data Manipulation Language) - 데이터 조작어	데이터를 저장, 검색, 수정, 삭제하는 명령어
DCL(Data Control Language) - 데이터 제어어	데이터 사용 권한과 관련된 명령어

SQL – DDL

DDL(데이터 정의어)

▷ 테이블 생성(만들기)

```
create table 테이블이름(
  name char(10),
  age int
 )
```

▷ 테이블 삭제

drop table 테이블 이름

truncate table 테이블 이름 – 모든 데이터 삭제(테이블은 유지)

▷ 테이블 변경

alter table 테이블 이름 **add** 칼럼추가

SQL - DML

DML(데이터 조작어)

▷ 자료 삽입(insert)

insert into 테이블이름(칼럼명) values (값1, 값2)

▷ 자료 조회(select)

select 칼럼명 from 테이블이름

▷ 자료 수정(update)

update 테이블이름 set='변경내용' where 칼럼명

▷ 자료 삭제(delete)

delete from 테이블 이름

SQL – DCL

DCL(데이터 제어어)

▷ 커밋과 롤백

트랜잭션(작은 업무 단위) 완료를 의미하는 명령어 - **commit**

변경사항을 취소하고 원래대로 복구하는 명령어 – **rollback**

▷ 권한 부여와 해제

DB 권한을 부여하는 명령어 - **grant**

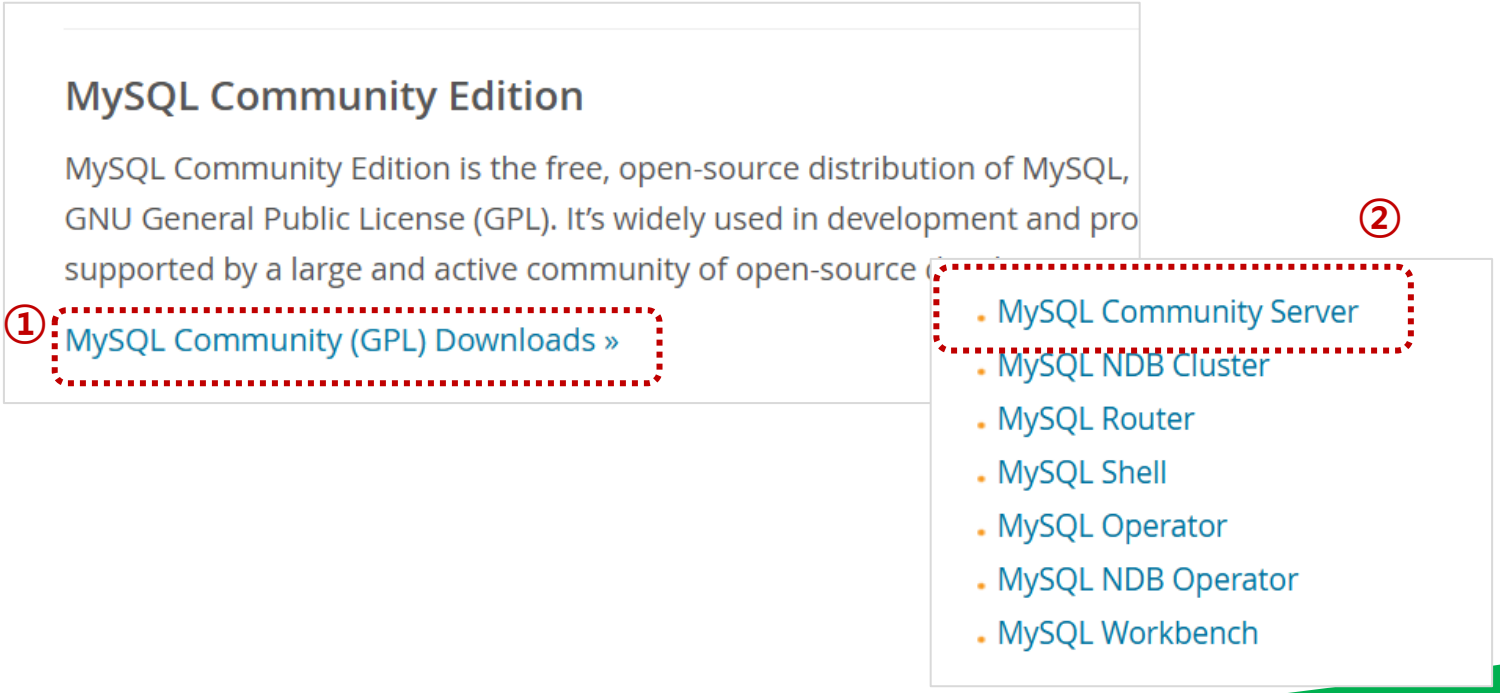
DB 권한을 해제하는 명령어 - **revoke**

MySQL DBMS

웹 애플리케이션에서 데이터베이스와의 연동은 필수적인 작업이다. 이런 상호작용을 위해서는 데이터베이스 관리 시스템이 설치되어야 한다.

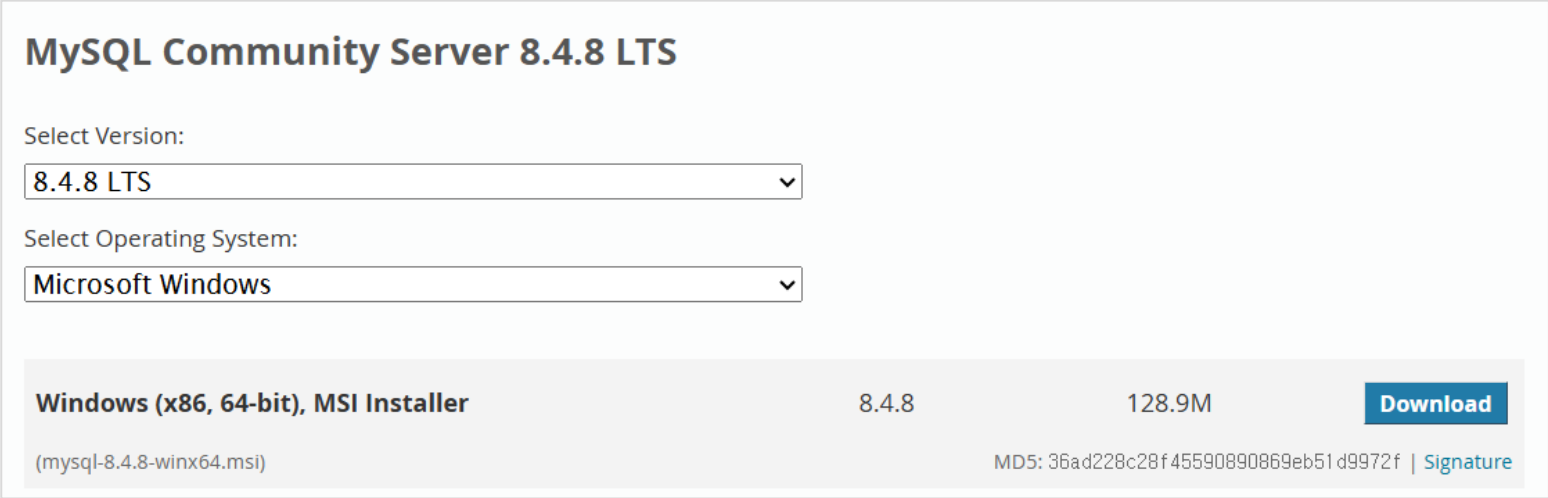
▷ MySQL 설치하기

mysql download 검색 > MySQL Community Downloads



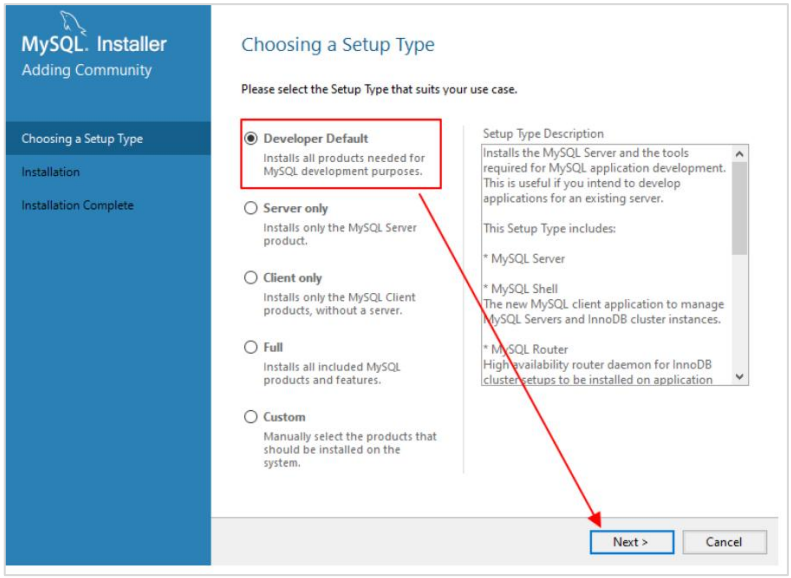
MySQL DBMS

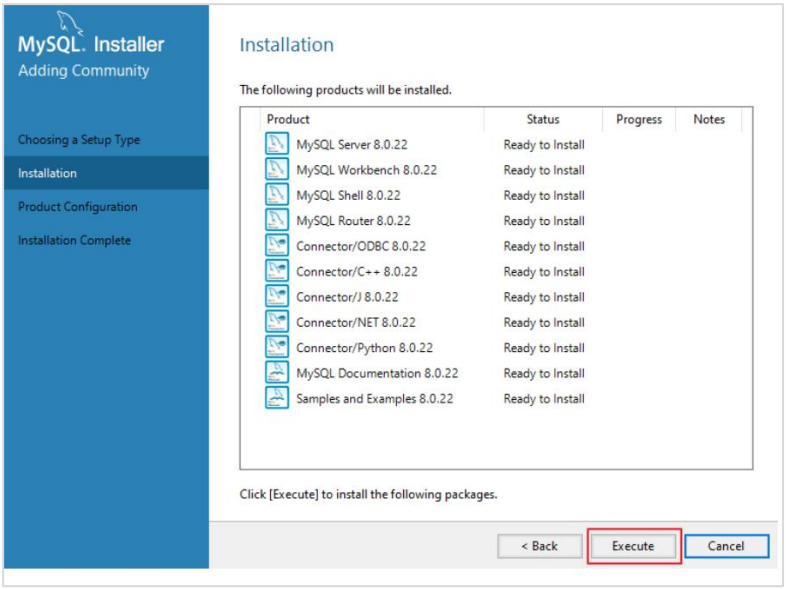
▷ MySQL 설치하기 – 8.4.8LTS 버전



MySQL DBMS

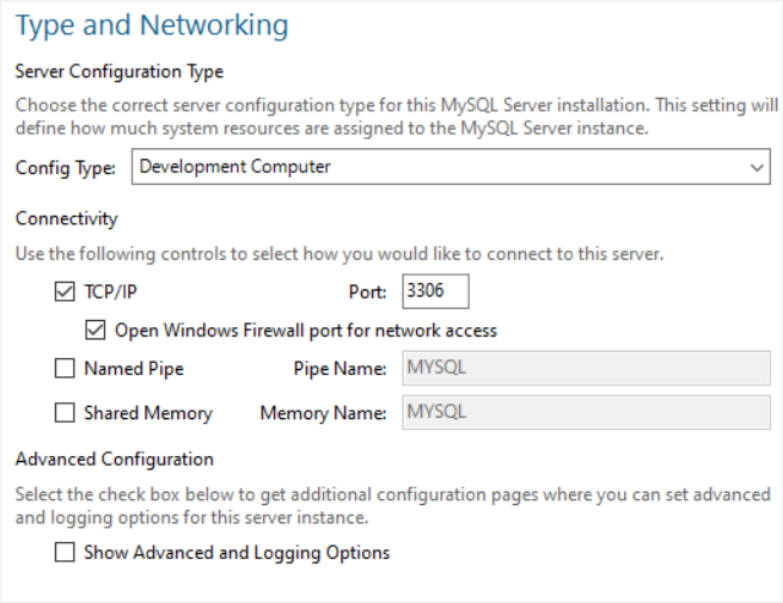
▷ MySQL 설치하기

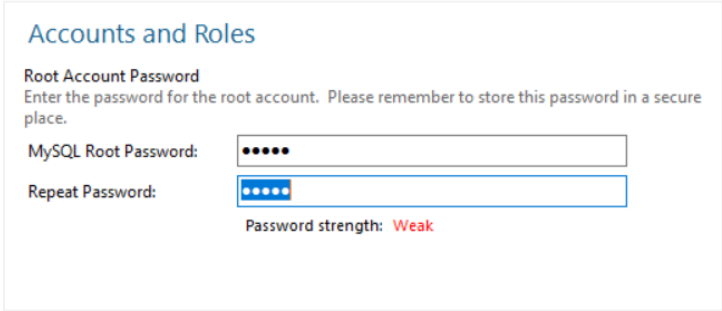




MySQL DBMS

▷ MySQL 설치하기





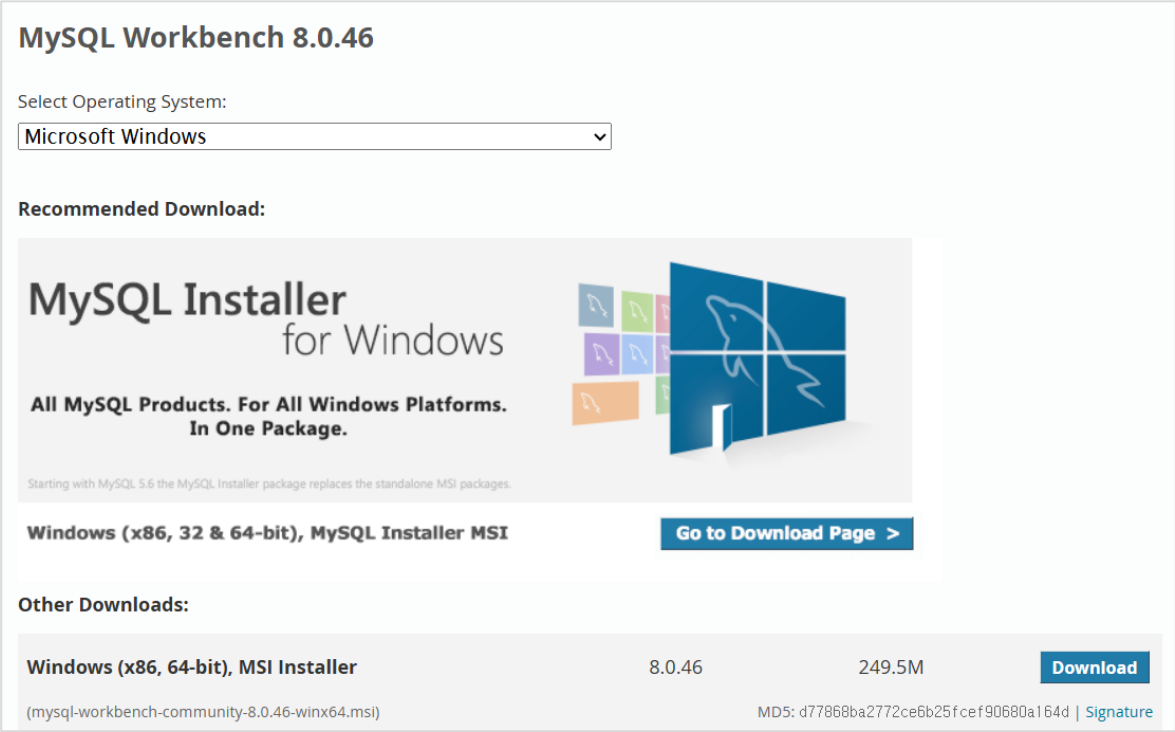
사용자 : root
비밀번호 : pw1234

포트 : 3306

MySQL WorkBench

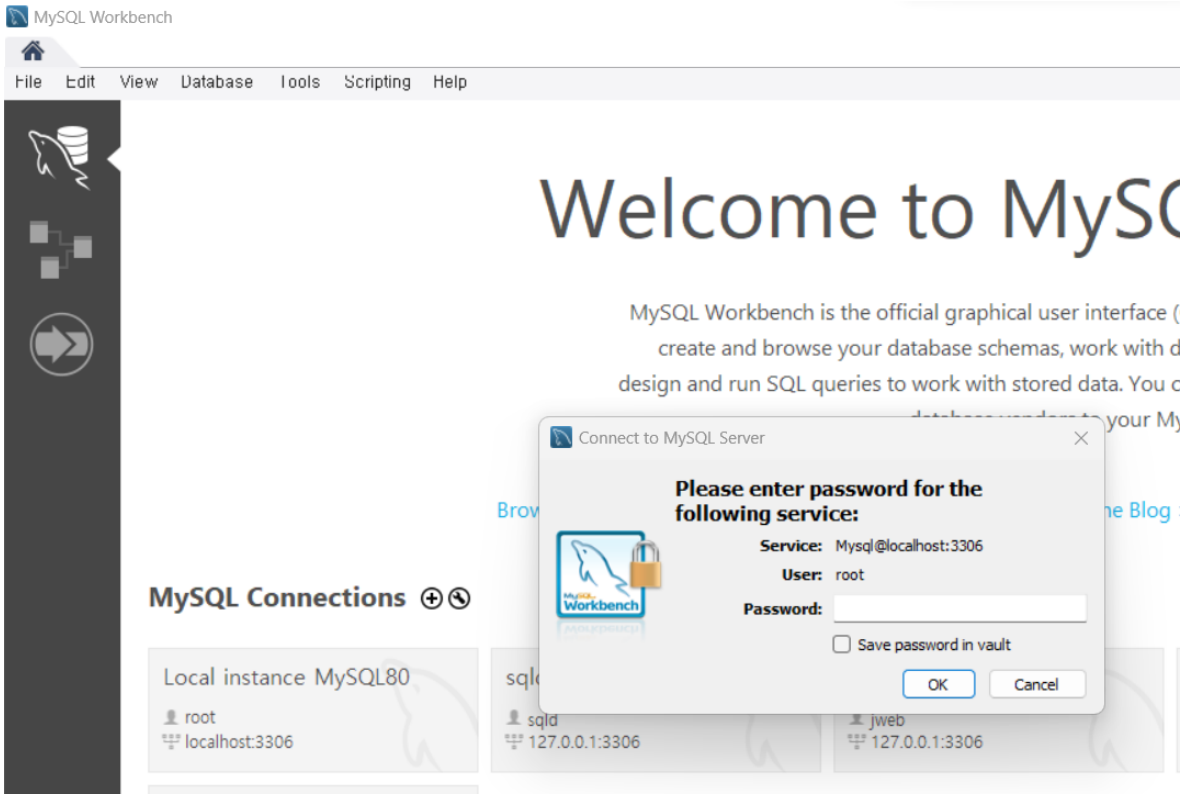
▷ **MySQL WorkBench(워크벤치)란?**

MySQL 데이터베이스를 시각적으로 관리하고 설계할 수 있도록 돕는 통합 개발 환경(IDE)입니다.



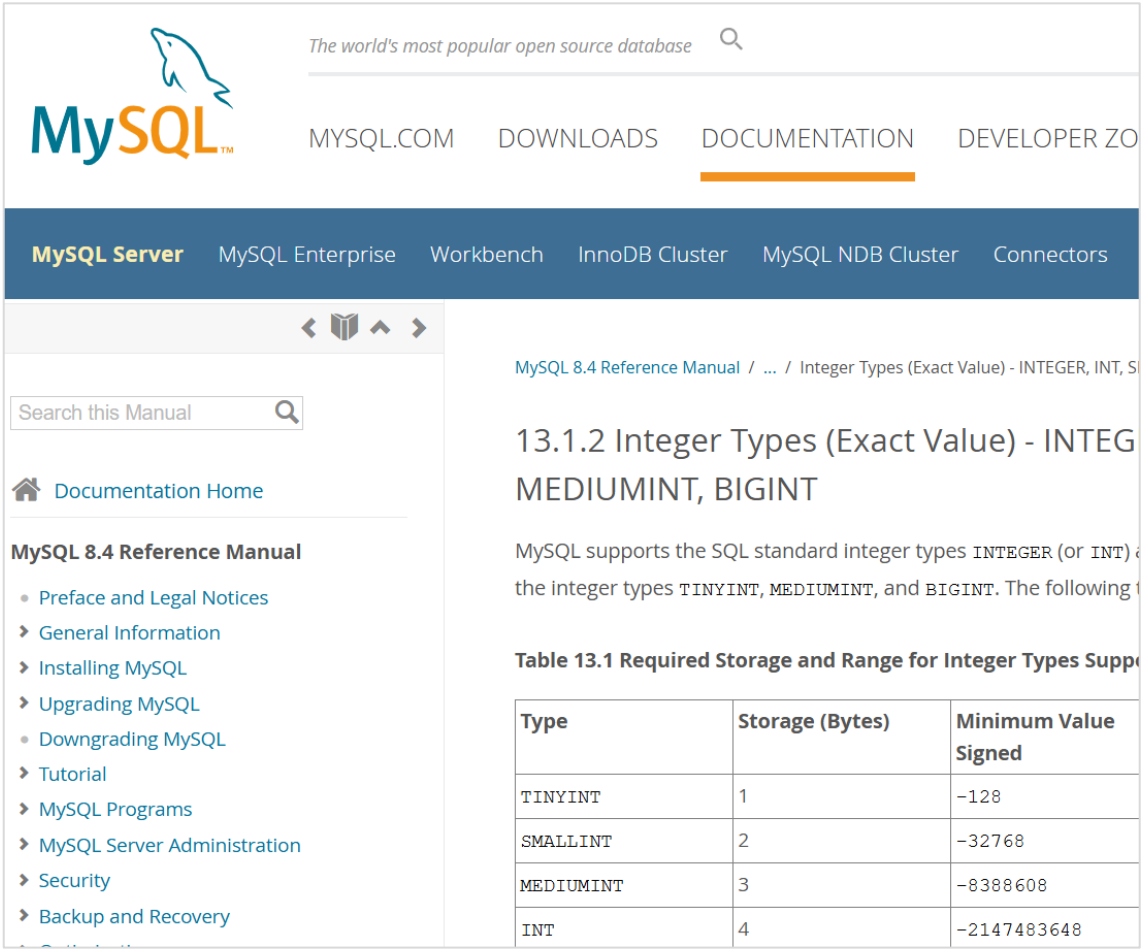
MySQL WorkBench

▷ 워크벤치 설치 Root 계정 접속



MySQL Data Type

- 데이터 타입(Data Type)



The world's most popular open source database

MYSQL.COM DOWNLOADS DOCUMENTATION DEVELOPER ZO

MySQL Server MySQL Enterprise Workbench InnoDB Cluster MySQL NDB Cluster Connectors

Search this Manual

Documentation Home

MySQL 8.4 Reference Manual

- Preface and Legal Notices
- General Information
- Installing MySQL
- Upgrading MySQL
- Downgrading MySQL
- Tutorial
- MySQL Programs
- MySQL Server Administration
- Security
- Backup and Recovery

MySQL 8.4 Reference Manual / ... / Integer Types (Exact Value) - INTEGER, INT, S

13.1.2 Integer Types (Exact Value) - INTEG
MEDIUMINT, BIGINT

MySQL supports the SQL standard integer types INTEGER (or INT) a
the integer types TINYINT, MEDIUMINT, and BIGINT. The following t

Table 13.1 Required Storage and Range for Integer Types Suppo

Type	Storage (Bytes)	Minimum Value Signed
TINYINT	1	-128
SMALLINT	2	-32768
MEDIUMINT	3	-8388608
INT	4	-2147483648

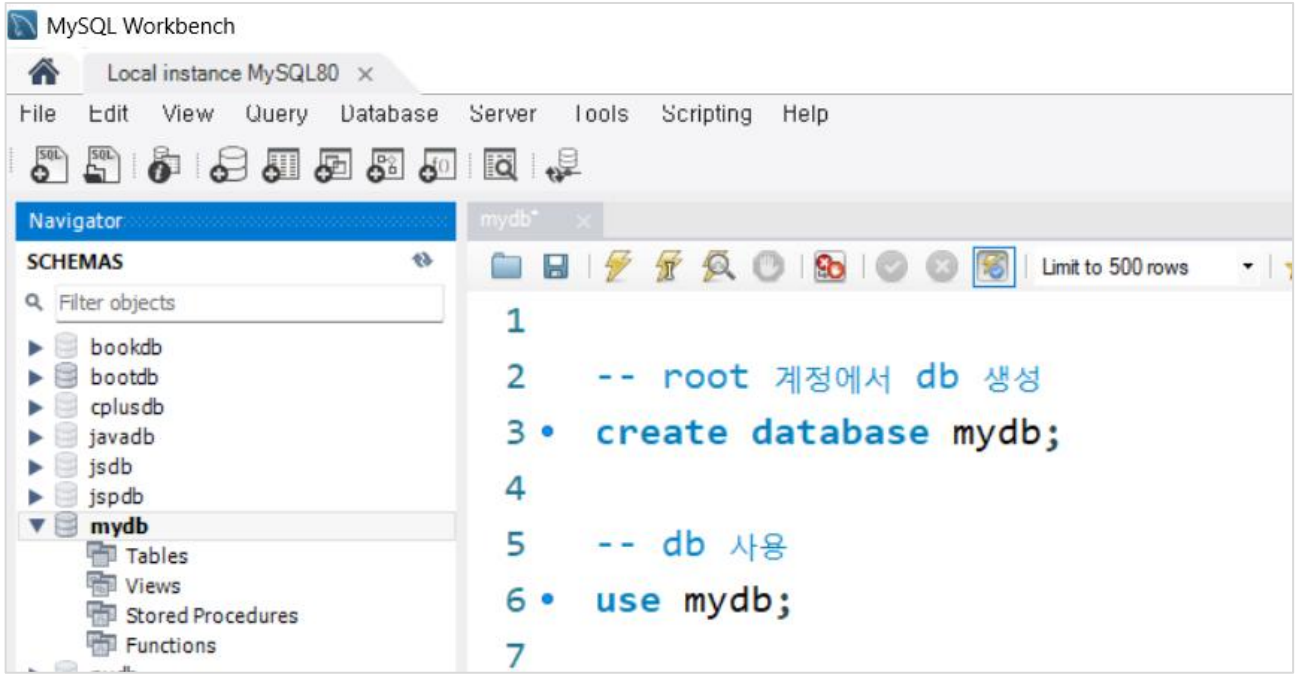
MySQL Data Type

- **데이터 타입(Data Type)**

데이터 타입	설명
CHAR	고정길이 문자, 최대 255byte, 디폴트값은 1byte
VARCHAR	가변길이 문자, 최대 65536byte, 디폴트값은 1byte
INT	가변 숫자, 십진수 기준 최대 220byte
TEXT	가변길이 문자, 최대 65536byte, 디폴트값은 1byte
DATE	날짜 – 연, 월, 일
TIMESTAMP	날짜 – 연, 월, 일, 시, 분, 초, 밀리초

DB 만들기

- **Root 계정에서 db 만들기**



테이블 만들기

- db에서 테이블 생성

```
-- 테이블 생성
create table test(
    msg varchar(30)
);

-- 자료 삽입
insert into test(msg) values ('Good Luck!');

-- 자료 검색
select msg from test;
```

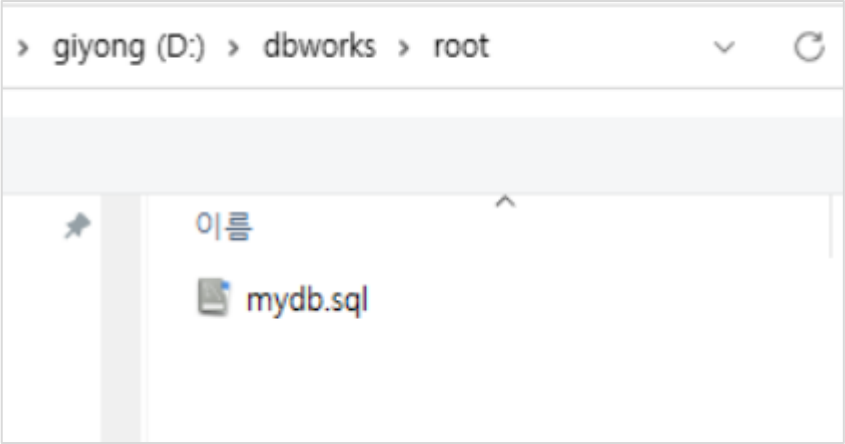
Result Grid | Filter Rows:

msg
Good Luck!

테이블 만들기

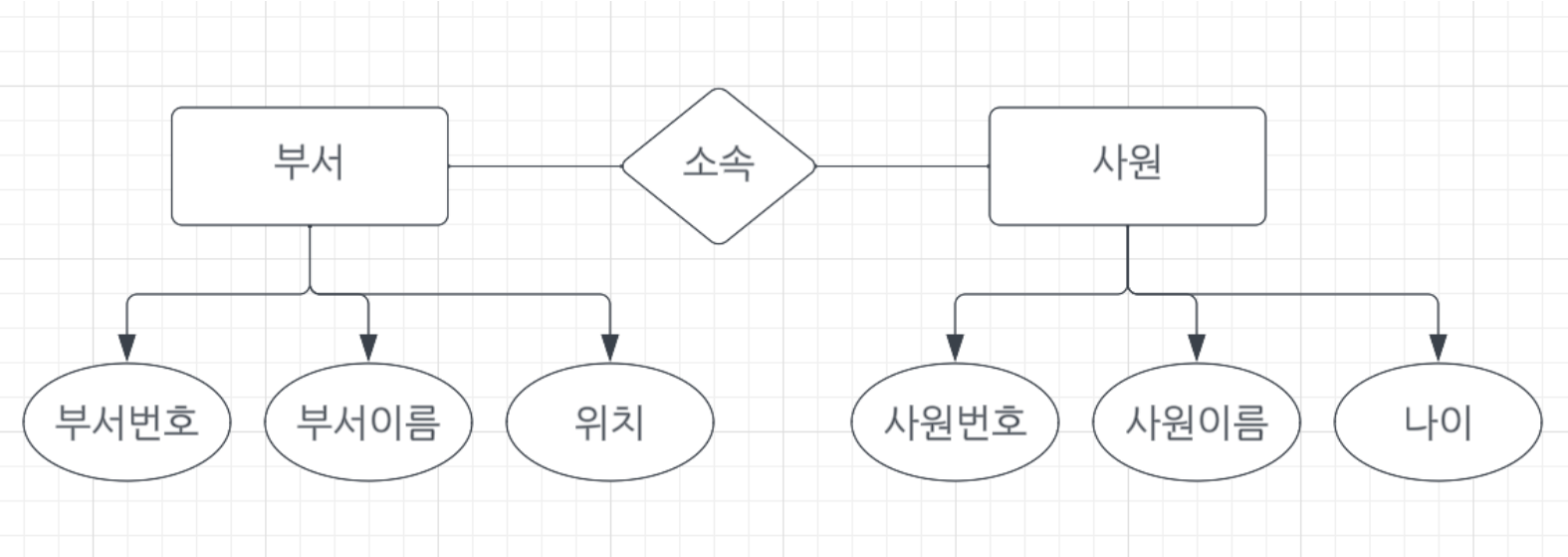
- **sql 파일 저장**

dbworks > root 폴더 생성



ER 다이어그램

- 부서와 사원의 개체 관계도



RDBMS(관계형 데이터베이스)

- **employee 테이블 생성**

```sql
-- 사원 테이블 생성
create table employee(
    id varchar(5),
    name varchar(20),
    salary int
);

-- 사원 추가
insert into employee(id, name, salary) values ('e1001', '김대리', 3500000);
insert into employee(id, name, salary) values ('e1002', '장그래', 2500000);
insert into employee(id, name) values ('e1002', '한강');
```

RDBMS(관계형 데이터베이스)

- **employee 테이블 생성**

```
/* 오류
 1. 기본키(primary key)가 없어서 고유하지 않음
 2. 필수 입력 제한이 없음(not null)
*/

-- 사원 전체 검색
select id, name, salary from employee;

-- 사원 1명 검색
select * from employee where id = 'e1002';

-- 테이블 삭제
drop table employee;
```

RDBMS(관계형 데이터베이스)

● 제약 조건

테이블들은 각 속성(칼럼)에 대한 무결성을 유지하기 위한 다양한 제약 조건(Constraints)이 적용되어 있다.

제약 조건에는 NOT NULL, 기본키, 외래키, CHECK 등이 있다.

✓ NOT NULL

칼럼을 정의할 때 NOT NULL 제약 조건을 명시하면 반드시 데이터를 입력해야 한다.

```
칼럼명  데이터 타입 NOT NULL
```

RDBMS(관계형 데이터베이스)

- **제약 조건**

  ✓ **기본키(PRIMARY KEY)**

  기본키는 Primay Key라고도 하며, UNIQUE와 NOT NULL 속성을 동시에 가진 제약 조건으로 테이블 당 1개의 기본키만 생성할 수 있다.

```
칼럼명  데이터 타입 PRIMARY KEY
또는
PRIMARY KEY(칼럼명, ...)
```

RDBMS(관계형 데이터베이스)

- **employee 테이블 생성**

```
-- 사원 테이블 생성
create table employee(
    id varchar(5) primary key,
    name varchar(20) not null,
    salary int
);

-- 사원 추가
insert into employee(id, name, salary) values ('e1001', '김대리', 3500000);
insert into employee(id, name, salary) values ('e1002', '장그래', 2500000);
-- insert into employee(id, name) values ('e1002', '한강');
-- Error Code: 1062. Duplicate entry 'e1002' for key 'employee.PRIMARY' 0
```

RDBMS(관계형 데이터베이스)

- **employee 테이블 생성**

```
insert into employee(id, name) values ('e1003', '한강');
-- insert into employee(id, salary) values ('e1004', 5000000);
-- Error Code: 1364. Field 'name' doesn't have a default value

-- 사원 검색
select * from employee;

-- 아이디가 'e1001'인 사원의 이름과 급여 검색
select name, salary from employee where id = 'e1001';

-- 급여가 300만원 미만인 사원의 모든 정보 검색
select * from employee where salary < 3000000;

-- 급여가 없는 사원의 정보 검색
select * from employee where salary is null;
```

RDBMS(관계형 데이터베이스)

- **employee 테이블 생성**

```sql
-- 급여가 많은 순으로 정렬하기 - 내림차순
select * from employee order by salary desc;

-- 급여가 작은 순으로 정렬하기 - 오름차순
select * from employee order by salary asc;

-- 사원 정보 수정('한강' 사원의 급여를 400만원 변경)
update employee set salary = 4000000 where id = 'e1003';

-- 사원 정보 삭제(이름이 '장그래'인 사원의 정보 삭제)
delete from employee where id = 'e1002';
```

id	name	salary
e1001	김대리	3500000
e1003	한강	4000000
NULL	NULL	NULL

API 서버 – DB 연결

- **프로젝트 개요**

프로젝트 이름 : api-server-db

깃허브 리포지터리: api-server-db

관련 주요 내용: README.md

개발 환경: node.js + express framework

데이터베이스: MySQL + Workbench(워크벤치)

API 서버 – DB 연결

- **README.md**

```
## 웹 서버 만들기
- Language: Javascript
- Node.js 설치
- package.json 생성: npm init -y

### Express 설치
- Express: Node.js 기반에서 동작하는 웹 서버 프레임워크
- 설치 : npm install express

### index.js 생성
- import express from 'express'
- 서버 코드 작성(PORT: 8000)
- 서버 실행 : node index.js
- 오류: ES6 문법 사용(import)
  package.json에 "type": "module"로 변경
- 서버 자동 실행 : nodemon 설치 : npm i nodemon
  package.json에 "type": "module"로 변경
```

API 서버 – DB 연결

- **README.md**

```
### mysql 라이브러리 설치
#### 설치
 - npm install mysql2

### mysql 설치 및 워크벤치 설정
- mysql 8.4
- workbench 설치

### root 계정에서 jsuser 계정 및 db 만들기
-- jsuser 계정 생성
create user jsuser@localhost identified by 'pwjs';

-- jsdb 생성
create database jsdb;

-- 권한 부여
grant all privileges on jsdb.* to jsuser@localhost;
```

API 서버 – DB 연결

- **README.md**

```
### jsdb에서 테이블 만들기
-- product 테이블 생성
create table product(
  id int auto_increment primary key,
    product_name varchar(20) not null,
    price int not null,
    created_at timestamp default current_timestamp
);

-- 상품 추가
insert into product(product_name, price) values('무선마우스', 30000);

-- 상품 검색
select * from product;
```

API 서버 – DB 연결

- **package.json**

```
{
  "name": "api-server(db)",
  "version": "1.0.0",
  "description": "- Language: Javascript\r - Node.js 설치\r
  "main": "index.js",
  ▷디버그
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "nodemon index.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module",
  "dependencies": {
    "express": "^5.2.1",
    "mysql2": "^3.20.0",
    "nodemon": "^3.1.14"
  }
}
```

API 서버 – DB 연결

- **index.js**

```
import express from 'express';
import mysql from 'mysql2';

const app = express();
const PORT = 8000;

// MySQL 연결 설정
const db = mysql.createConnection({
  host: 'localhost',
  user: 'jsuser',
  password: 'pwjs',
  database: 'jsdb'
});

// MySQL 연결 확인
db.connect((err) => {
  if (err) {
    console.error('Error connecting to MySQL:', err);
    return;
  }
  console.log('MySQL에 연결되었습니다.');
});
```

API 서버 – DB 연결

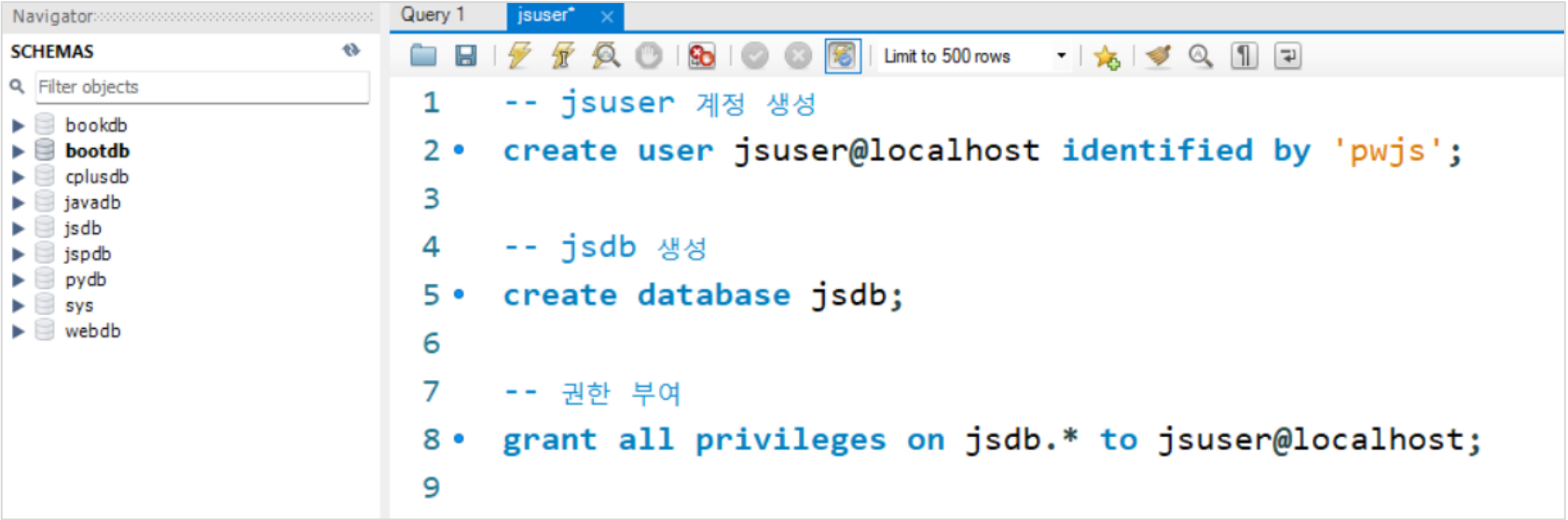
- **index.js**

```
// 라우트 설정
app.get('/', (req, res) => {
  res.send('Hello, World!');
});

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

```
PS D:\웹(js기반)\api-server\api-server(db)> nodemon index
[nodemon] 3.1.11
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
Server is running on port 8000
MySQL에 연결되었습니다.
```

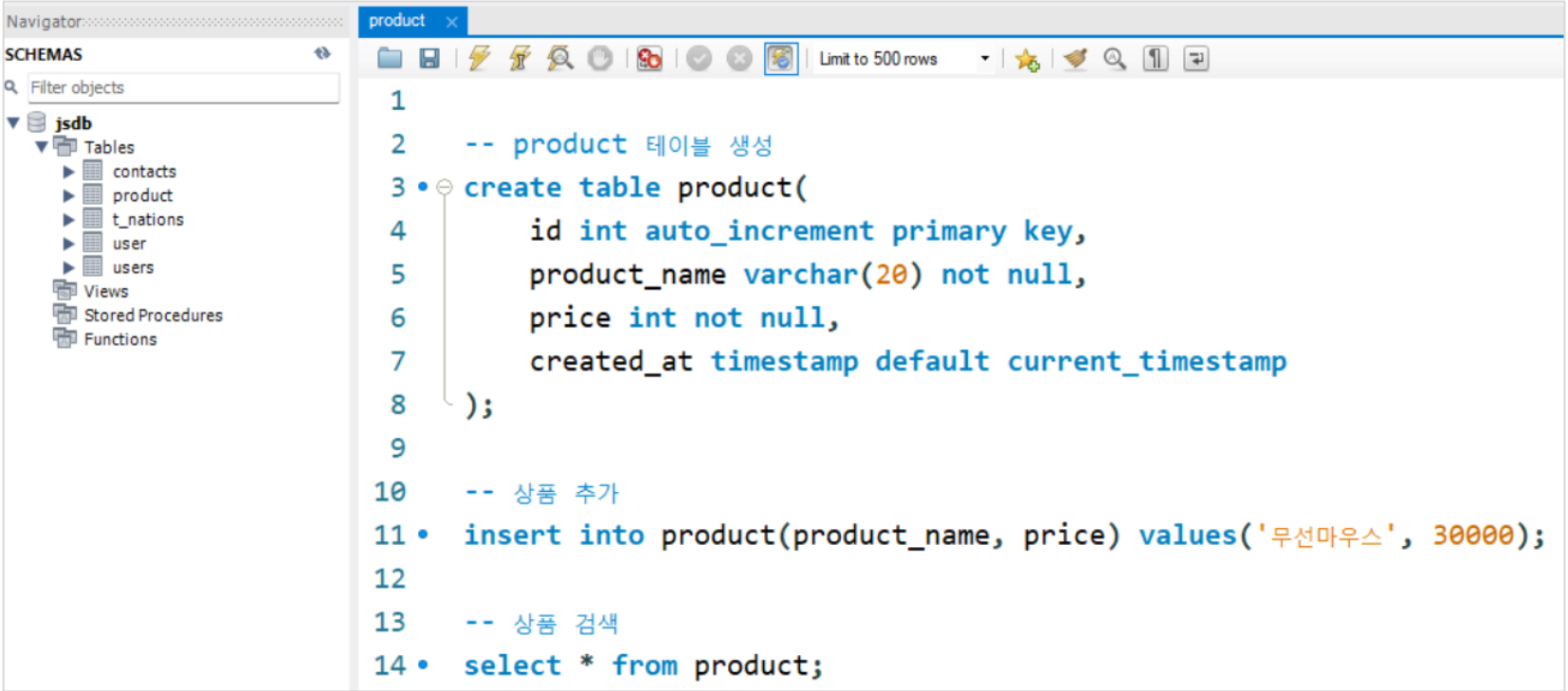
API 서버 – DB 연결

- **Workbench > ROOT**



API 서버 – DB 연결

- **Workbench > JSUSER**



API 서버 – DB 연결

- **상품 목록 조회**

```
//상품 목록 조회 라우트 설정
app.get('/products', (req, res) => {
  const sql = 'SELECT * FROM product';
  db.query(sql, (err, results) => {
    if (err) {
      // 에러 발생 시 500 상태 코드와 에러 메시지 반환
      console.error('상품 조회 에러:', err);
      res.status(500).send('상품 조회 중 오류가 발생했습니다.');
      return; // 즉시 종료
    }
    res.json(results); // 성공 시 상품 목록을 JSON 형식으로 반환
  });
});
```

API 서버 – DB 연결

- 상품 목록 조회 - 포스트맨

Body 200 OK 37 ms 429 B

{} JSON Preview Visualize


```
[
    {
        "id": 1,
        "product_name": "무선마우스",
        "price": 30000,
        "created_at": "2026-04-06T04:11:07.000Z"
    },
    {
        "id": 2,
        "product_name": "유선키보드",
        "price": 20000,
        "created_at": "2026-04-06T05:00:49.000Z"
    }
]
```

API 서버 – DB 연결

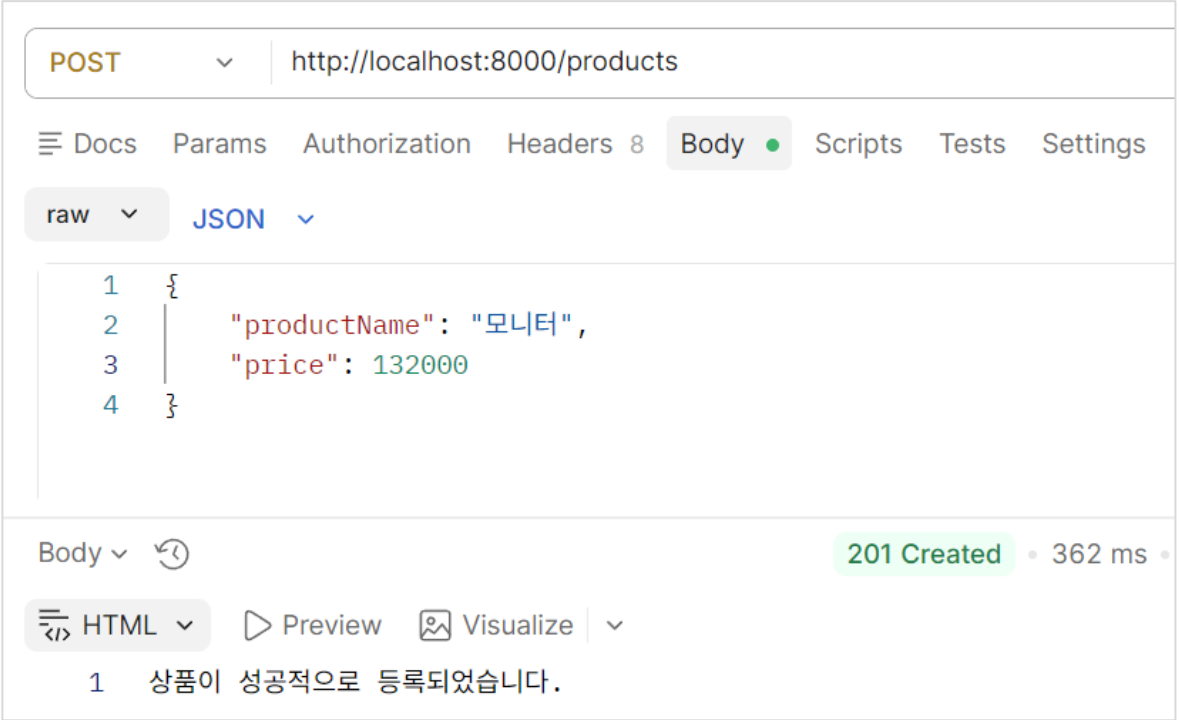
- **상품 등록**

```
// JSON 요청 본문을 파싱하기 위한 미들웨어 설정
app.use(express.json());

// 상품 등록 페이지 요청 라우트 설정
app.post('/products', (req, res) => {
  const { productName, price } = req.body; // 요청 본문에서 상품 이름과 가격 추출
  const sql = 'INSERT INTO product (product_name, price) VALUES (?, ?)';
  db.query(sql, [productName, price], (err, results) => {
    if (err) {
      // 에러 발생 시 500 상태 코드와 에러 메시지 반환
      console.error('상품 등록 에러:', err);
      res.status(500).send('상품 등록 중 오류가 발생했습니다.');
      return; // 즉시 종료
    }
    res.status(201).send('상품이 성공적으로 등록되었습니다.');
  });
});
```

API 서버 – DB 연결

- **상품 등록 - 포스트맨**

상품을 등록하면 id와 날짜는 자동 생성됨



API 서버 – DB 연결

- **상품 상세보기**

```
// 상품 상세 조회 라우트 설정
app.get('/products/:id', (req, res) => {
  const productId = req.params.id; // URL 경로에서 상품 ID 추출
  const sql = 'SELECT * FROM product WHERE id = ?';
  db.query(sql, [productId], (err, results) => {
    if (err) {
      console.error('상품 조회 에러:', err);
      res.status(500).send('상품 조회 중 오류가 발생했습니다.');
      return;
    }
    res.json(results[0]); // 첫 번째 결과만 반환
  });
});
```

API 서버 – DB 연결

- **상품 상세보기 - 포스트맨**

id가 2번인 제품 상세 보기

GET http://localhost:8000/products/2

Docs | Params | Authorization | Headers 8 | Body | Scripts | Tests | Settings

Query Params

Key	Value	Descript
Key	Value	Descript

Body 200 OK 33 ms 330 B

{} JSON | Preview | Visualize


```
1 {
2     "id": 2,
3     "product_name": "유선키보드",
4     "price": 20000,
5     "created_at": "2026-04-07T00:57:37.000Z"
6 }
```

API 서버 – DB 연결

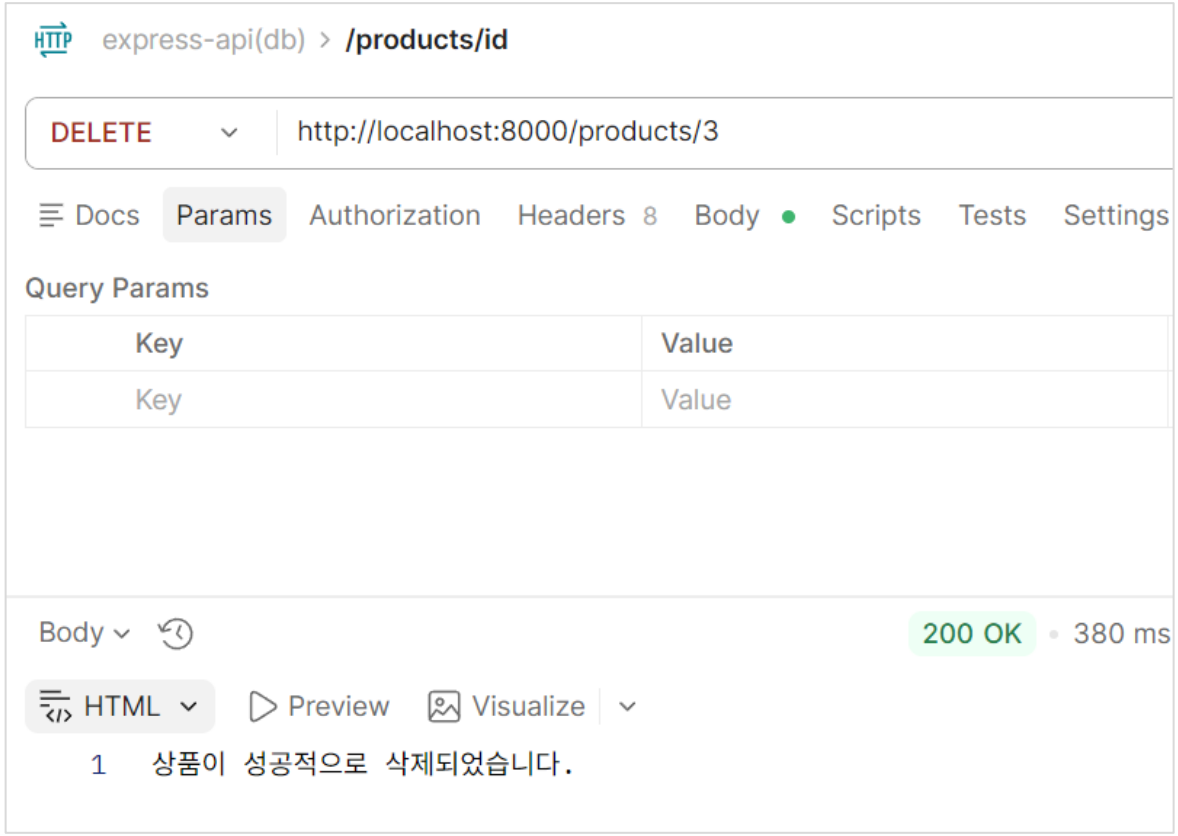
- **상품 삭제**

```
// 상품 삭제 라우트 설정
app.delete('/products/:id', (req, res) => {
  const productId = req.params.id; // URL 경로에서 상품 ID 추출
  const sql = 'DELETE FROM product WHERE id = ?';
  db.query(sql, [productId], (err, results) => {
    if (err) {
      console.error('상품 삭제 에러:', err);
      res.status(500).send('상품 삭제 중 오류가 발생했습니다.');
      return;
    }
    res.send('상품이 성공적으로 삭제되었습니다.');
  });
});
```

API 서버 – DB 연결

- **상품 삭제 - 포스트맨**

Id가 3번인 제품 삭제

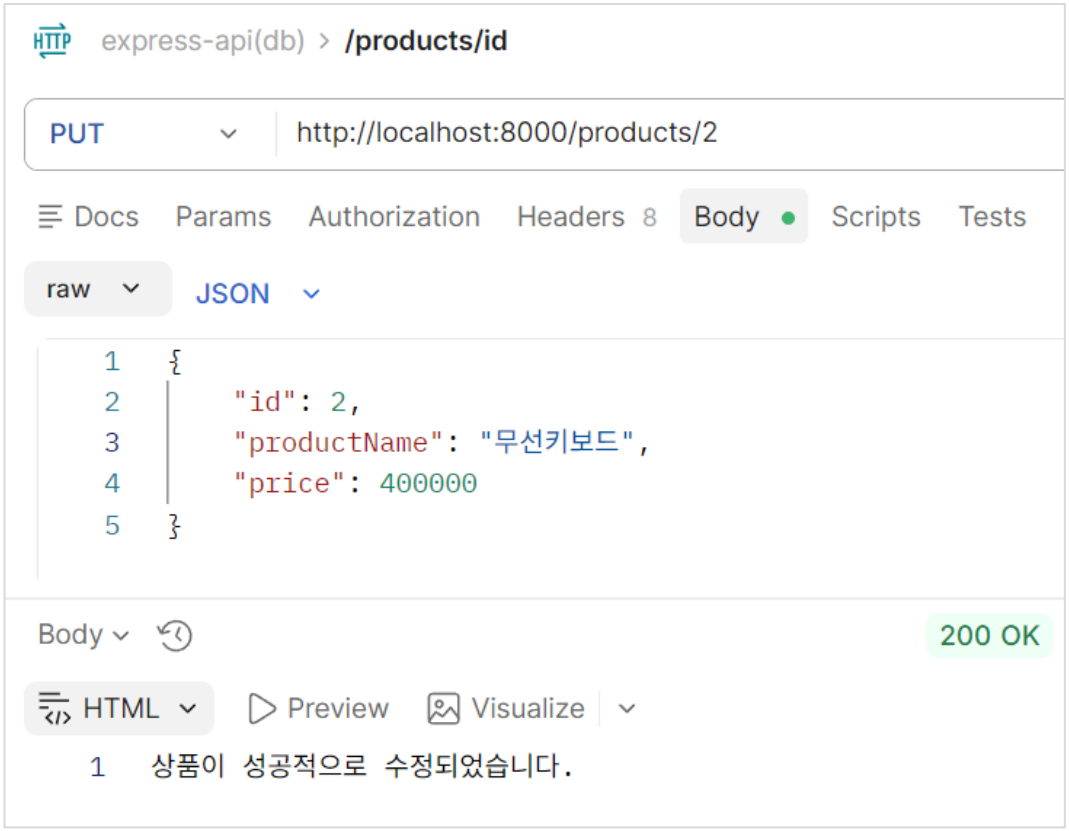


API 서버 – DB 연결

- **상품 수정**

```
// 상품 수정 라우트 설정
app.put('/products/:id', (req, res) => {
  const productId = req.params.id;
  const { productName, price } = req.body;
  const sql = 'UPDATE product SET product_name = ?, price = ? WHERE id = ?';
  db.query(sql, [productName, price, productId], (err, results) => {
    if (err) {
      console.error('상품 수정 에러:', err);
      res.status(500).send('상품 수정 중 오류가 발생했습니다.');
      return;
    }
    res.send('상품이 성공적으로 수정되었습니다.');
  });
});
```

API 서버 – DB 연결

- **상품 수정 - 포스트맨**

id 2번인 제품 ' 유선 키보드'를 '무선 키보드'로 가격은 40000원으로 변경